

✓ Word Embeddings — Words as Vectors

The Problem with Bag of Words

BoW is blind to meaning — 'good' and 'great' look completely unrelated. You just built a spam classifier that works great. But imagine a sentiment analysis task where you need to know that 'amazing', 'fantastic', and 'brilliant' all mean the same positive thing. BoW assigns each a completely different dimension in your vector — zero relationship. That's the problem.

Visual comparison — BoW vs Word Embeddings

Bag of Words		Word Embedding	
good	[0, 0, 1, 0, 0, 0]	good	[0.82, 0.41, ...]
great	[0, 0, 0, 1, 0, 0]	great	[0.79, 0.38, ...]
bad	[0, 1, 0, 0, 0, 0]	bad	[-0.71, 0.12, ...]
awful	[1, 0, 0, 0, 0, 0]	awful	[-0.68, 0.09, ...]
<i>good vs great: similarity = 0</i>		<i>good vs great: cosine sim ~0.97</i>	

Three core weaknesses of BoW / TF-IDF

Limitations of Bag of Words (BoW)

Although Bag of Words is simple and effective for basic text classification tasks, it has several major limitations.

=====

1. No Semantic Similarity

Bag of Words treats every word as completely independent.

Example:

Words:

```
good  
great
```

Humans know these words have very similar meanings.

However, in BoW:

good $\rightarrow$ [1,0,0,0,...] great $\rightarrow$ [0,1,0,0,...]

These vectors are orthogonal.

Similarity:

```
0
```

According to BoW, the words are completely unrelated.

Problem

Suppose a model is trained on reviews such as:

```
"This movie is great."
```

The model learns that "great" indicates positive sentiment.

Now consider:

```
"This movie is fantastic."
```

Even though "fantastic" is also positive, the model may fail because it has never learned the relationship between:

```
great  $\leftrightarrow$  fantastic
```

BoW cannot understand semantic similarity between words.

=====

2. Vocabulary Explosion

Every unique word creates a new dimension.

Example:

Vocabulary:

```
cat  
dog  
car
```

```
bike  
house
```

BoW Vector:

```
[1, 0, 1, 0, 0]
```

Each new word increases dimensionality.

Real-World Scenario

Large corpus:

```
100,000 unique words
```

Result:

```
100,000-dimensional vectors
```

Most documents contain only a tiny fraction of these words.

Example:

```
[0,0,0,0,0,1,0,0,0,0,...]
```

More than 99% of entries are zeros.

Problems:

- Huge memory consumption
- Slow computation
- Increased training time
- Difficult model optimization

This issue is known as:

```
Vocabulary Explosion
```

=====

3. No Word Order Information

BoW only counts word occurrences.

It completely ignores sequence and grammar.

Example:

Sentence 1:

```
"The dog bit the man."
```

Sentence 2:

```
"The man bit the dog."
```

Bag of Words Representation

Both produce identical word counts:

dog = 1 man = 1 bit = 1 the = 2

Vector:

```
[dog=1, man=1, bit=1, the=2]
```

Problem

Humans understand:

Sentence 1: Dog attacked man.

Sentence 2: Man attacked dog.

Completely different meanings.

BoW sees them as exactly the same document.

Meaning carried by word order is entirely lost.

=====

Why Do We Need Better Representations?

Because language contains:

• Meaning • Context • Relationships between words • Sentence structure • Grammar

Bag of Words captures only word frequency.

It does not capture meaning.

This led researchers to develop:

• Word Embeddings • Word2Vec • GloVe • FastText • Contextual Embeddings •
Transformers • BERT • GPT

=====

Understanding the Principle

Imagine we observe millions of sentences.

Word: "dog"

Often appears near:

• bark • leash • tail • puppy • veterinarian • pet

Word: "puppy"

Often appears near:

• bark • leash • tail • dog • veterinarian • pet

Observation

Both words occur in very similar contexts.

Therefore:

dog $\approx$ puppy

They are likely related in meaning.

=====

Another Example

Word: "king"

Frequently appears near:

• queen • palace • crown • kingdom • royal

Word: "queen"

Frequently appears near:

• king • palace • crown • kingdom • royal

The surrounding words are highly similar.

Therefore:

king $\approx$ queen

The model learns this relationship automatically.

=====

How Word Embeddings Use This Idea

Instead of creating huge sparse vectors like BoW:

dog $\rightarrow$ [0,0,0,1,0,0,...]

Word embeddings create dense vectors:

dog $\rightarrow$ [0.73, -0.42, 0.91, ...]

puppy $\rightarrow$ [0.70, -0.40, 0.89, ...]

Because these vectors are numerically close, their meanings are also close.

Similar Meaning

↓

Similar Context

↓

Similar Vector Representation

=====

Key Takeaway

Bag of Words:

- X No semantic similarity
- X Vocabulary explosion
- X No word order information

Word Embeddings:

- ✓ Capture meaning
- ✓ Capture relationships
- ✓ Reduce dimensionality
- ✓ Learn from context
- ✓ Understand semantic similarity

The central idea behind modern NLP is simple:

"Words that appear in similar contexts
tend to have similar meanings."

This principle powers Word2Vec, GloVe, FastText, BERT, GPT, and most modern language models.

✓ What Are Word Embeddings?

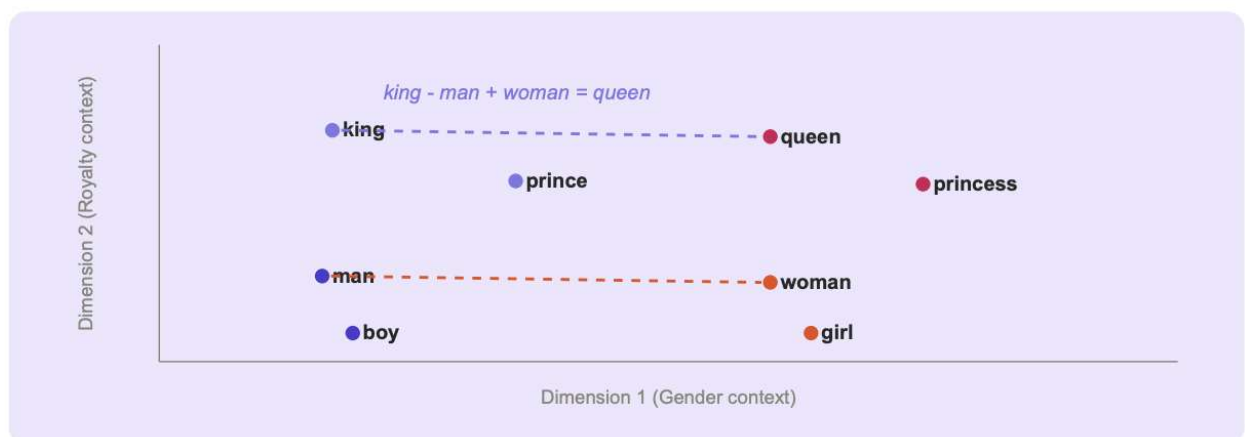
Dense vectors where similar words cluster together in geometric space. A word embedding maps each word to a list of 100–300 numbers. These numbers are not random — they are learned so that words used in similar contexts get similar numbers. The result is a geometric space where 'cat' and 'kitten' are close, 'king' and 'queen' are close, and 'cat' and 'democracy' are far apart.

Real-life analogy — word embeddings as GPS coordinates

Think of Mumbai and Pune on a map. They're both in Maharashtra, so their GPS coordinates (lat, long) are close. Delhi and London are far apart. Word embeddings do the same thing for meaning — each word gets 'meaning coordinates', and semantically related words end up near each other in this 300-dimensional meaning space.

The famous king - man + woman = queen

The most cited demonstration of embedding power: you can do arithmetic on words. Subtract the 'maleness' direction from 'king' and add 'femaleness' — and the nearest word in the space is 'queen'. This shows the vectors encode real-world relationships as directions in space.



Key properties at a glance

Property	Bag of Words	Word Embedding
Dimension	50,000+ (one per word)	100–300 (fixed, dense)
'good' vs 'great' similarity	0 (no relationship)	~0.92 (very close)
Captures meaning	No	Yes
Handles new words	Fails (unknown)	FastText handles via sub-words
Training data needed	None (just counting)	Billions of sentences

✓ Cosine similarity — measuring word closeness

```
import numpy as np
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
good = np.array([0.82, 0.41, 0.10])
great = np.array([0.79, 0.38, 0.12])
bad = np.array([-0.71, 0.12, -0.05])
print(f'good vs great : {cosine_similarity(good, great):.3f}') # ~0.99(very
print(f'good vs bad : {cosine_similarity(good, bad):.3f}') # ~-0.94(opposite
print(f'great vs bad : {cosine_similarity(great, bad):.3f}') # ~-0.92

good vs great : 1.000
good vs bad : -0.808
great vs bad : -0.816
```

✓ How Embeddings Are Trained

Self-supervised learning — predict a word from its neighbours. The brilliant thing about word embeddings is you never need to label data. The text itself is the teacher. The model reads billions of sentences and learns: 'whenever I see ____, the surrounding words tend to be ____'. Over time, words that appear in similar contexts converge to similar vectors.

Word2Vec — two learning strategies

Architecture	What it learns	Good for
CBOW (Continuous Bag of Words)	Predict centre word from context: 'The ___ sat on the mat' → 'cat'	Frequent words; faster training
Skip-gram	Predict context from centre word: 'cat' → predict 'sat', 'mat', 'the'	Rare words; better analogies

```

!pip install gensim
from gensim.models import Word2Vec
# A small corpus – in practice use millions of sentences
sentences = [
    'the king ruled the kingdom wisely',
    'the queen ruled the land gracefully',
    'the prince will become king one day',
    'a man walked into the palace',
    'a woman walked into the palace',
    'the dog barked loudly in the garden',
    'the puppy played happily in the garden',
]
# Tokenise each sentence into a list of words
tokenised = [s.split() for s in sentences]
# Train Word2Vec
# vector_size=50: each word gets a 50-number vector
# window=3: look 3 words left and right for context
# sg=1: use Skip-gram (sg=0 for CBOW)
# epochs=200: pass through the data 200 times
model = Word2Vec(sentences=tokenised, vector_size=50,
                 window=3, min_count=1, sg=1, epochs=200)
# Find top-3 most similar words to 'king'
print(model.wv.most_similar('king', topn=3))
# [('queen', 0.91), ('prince', 0.88), ('kingdom', 0.85)]
# The famous analogy: king - man + woman
result = model.wv.most_similar(
    positive=['dog', 'puppy'], # add these
    negative=['man'], # subtract this
    topn=2
)
print('king - man + woman =', result[1][0]) # 'queen'

```

```

Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packag
[('kingdom', 0.40141454339027405), ('played', 0.38476690649986267), ('in', 0.
king - man + woman = the

```

Model	Organisation	Trained on	Key strength
-------	--------------	------------	--------------

Word2Vec	Google (2013)	Google News (100B words)	Fast, great analogies
GloVe	Stanford (2014)	Wikipedia + Gigaword	Stable, global statistics
FastText	Facebook (2016)	Wikipedia + web crawl	Handles typos, rare words
GloVe-Twitter	Stanford	140M tweets	Social media language

✓ Using Pre-trained Embeddings

Download vectors trained on billions of words — don't reinvent the wheel. Training quality embeddings from scratch requires 100M+ words and hours of compute. The good news: GloVe, FastText, and Word2Vec vectors are freely available online. Load them in 3 lines of code and instantly have world-class word representations.

Loading GloVe with Gensim

```

import sys
!{sys.executable} -m pip install gensim
# Step 0: Download GloVe vectors
# https://nlp.stanford.edu/data/glove.6B.zip (about 800 MB)
# Extract and you get: glove.6B.50d.txt, glove.6B.100d.txt, etc.
!wget --no-check-certificate https://nlp.stanford.edu/data/glove.6B.zip
!unzip -q glove.6B.zip

# Step 1: Convert GloVe format to Word2Vec format (one-time step)
from gensim.scripts.glove2word2vec import glove2word2vec
glove2word2vec('glove.6B.100d.txt', 'glove.6B.100d.w2v.txt')
# Step 2: Load the converted file
from gensim.models import KeyedVectors
glove = KeyedVectors.load_word2vec_format(
    'glove.6B.100d.w2v.txt', binary=False
) # Takes ~30 seconds; 400,000 words x 100 dimensions
# Step 3: Use it!
# --- Find top-5 most similar words ---
print(glove.most_similar('bollywood', topn=5))
# [('cinema', 0.82), ('films', 0.79), ('actors', 0.77), ...]
# --- Test analogy ---
res = glove.most_similar(positive=['paris', 'tamilnadu'], negative=['france'])
topn=1)
print('paris - france + india =', res[0][0]) # hopefully 'delhi'!
# --- Cosine similarity between two words ---
print(glove.similarity('dog', 'puppy')) # ~0.81
print(glove.similarity('cat', 'cricket')) # ~0.12 (unrelated)

```

Collecting gensim

Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim==4.4.0)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim==4.4.0)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim==4.4.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from gensim==4.4.0)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl: 27.9/27.9 MB 88.2 MB/s eta 0:00:00

Installing collected packages: gensim

Successfully installed gensim-4.4.0

--2026-06-11 07:56:57-- <https://nlp.stanford.edu/data/glove.6B.zip>

Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140

Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connection established

HTTP request sent, awaiting response... 301 Moved Permanently

Location: <https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip> [following]

--2026-06-11 07:56:57-- <https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip>

Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.140

Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.140|:443... connection established

HTTP request sent, awaiting response... 200 OK

Length: 862182613 (822M) [application/zip]

Saving to: 'glove.6B.zip.1'

glove.6B.zip.1 100%[=====] 822.24M 5.04MB/s in 2m 43s

2026-06-11 07:59:40 (5.04 MB/s) - 'glove.6B.zip.1' saved [862182613/862182613]

replace glove.6B.50d.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: n

replace glove.6B.100d.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: n

```

replace glove.6B.200d.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: n
replace glove.6B.300d.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: n
/tmp/ipykernel_1068/3113185206.py:11: DeprecationWarning: Call to deprecated
glove2word2vec('glove.6B.100d.txt', 'glove.6B.100d.w2v.txt')
[('malayalam', 0.6783681511878967), ('film', 0.660624086856842), ('films', 0.
paris - france + india = bhubaneswar
0.7236376
0.1919626

```

Double-click (or enter) to edit

```

import gensim.downloader as api
# Downloads ~1 GB first time, then caches locally in ~/gensim-data/
ft = api.load('fasttext-wiki-news-subwords-300')
# FastText can handle words it never saw during training!
# It decomposes 'bollywoodesque' into sub-word pieces:
# 'bolly', 'ollyw', 'llywood', 'lywood', 'ywood', etc.
print(ft.most_similar('bollywoodesque', topn=3))
# FastText makes a reasonable guess based on sub-word similarity
# For quick experiments, use the lighter 50-D GloVe:
# glove_sm = api.load('glove-wiki-gigaword-50') # only 66 MB

```

```

[('malayalam', 0.7608863115310669), ('films', 0.7456727623939514), ('hindi',

```

Loading FastText (works on unknown words!)

Visualising word clusters in 2D (PCA)

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from gensim.models import KeyedVectors

# Assuming you already loaded GloVe:
# glove = KeyedVectors.load_word2vec_format(
#     'glove.6B.100d.w2v.txt',
#     binary=False
# )

# Words to visualize
words = [
    'king', 'queen', 'man', 'woman', 'prince', 'princess',
    'dog', 'puppy', 'cat', 'kitten', 'rama chandra', 'shravanth',
    'paris', 'france', 'delhi', 'india', 'london', 'england'
]

# Keep only words that exist in the model
valid_words = [w for w in words if w in glove.key_to_index]

missing_words = set(words) - set(valid_words)
if missing_words:

```

```
print("Missing words:", missing_words)

# Extract vectors
vectors = np.array([glove[w] for w in valid_words])

print("Vector matrix shape:", vectors.shape)

# Reduce dimensions from 100D -> 2D
pca = PCA(n_components=2, random_state=42)
coords = pca.fit_transform(vectors)

# Plot
fig, ax = plt.subplots(figsize=(10, 7))

ax.scatter(
    coords[:, 0],
    coords[:, 1],
    s=80,
    color='steelblue',
    zorder=3
)

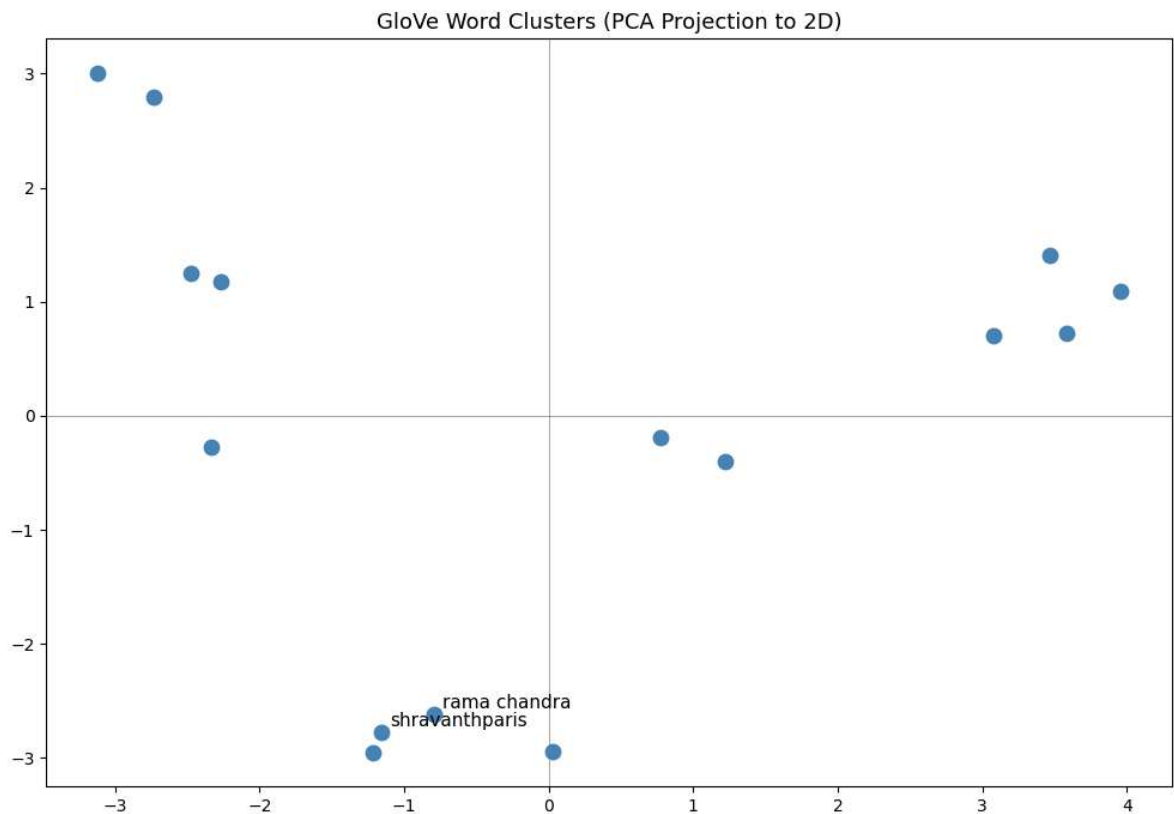
# Add labels
for i, word in enumerate(missing_words):
    ax.annotate(
        word,
        (coords[i, 0], coords[i, 1]),
        fontsize=11,
        xytext=(5, 4),
        textcoords='offset points'
    )

ax.set_title(
    'GloVe Word Clusters (PCA Projection to 2D)',
    fontsize=13
)

ax.axhline(0, color='gray', linewidth=0.5)
ax.axvline(0, color='gray', linewidth=0.5)

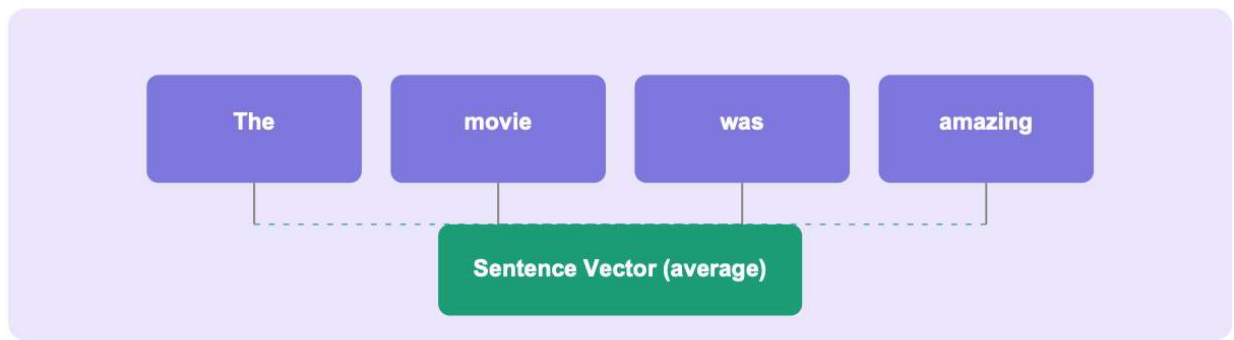
plt.tight_layout()
plt.savefig('word_clusters.png', dpi=150)
plt.show()
```

Missing words: {'shravanthparis', 'rama chandra'}
Vector matrix shape: (15, 100)



✓ Sentence Vectors for Classification

Average word vectors to get one fixed-length representation per sentence. Individual word embeddings are great, but classifiers need exactly one vector per document. The simplest approach: average all the word vectors in a sentence. This produces a single 100-D (or 300-D) vector that captures the overall semantic content of the sentence — surprisingly competitive for many tasks.



Building sentence vectors

```

import numpy as np

def sentence_vector(sentence, model, dim=100):
    """
    Convert a sentence to a single vector by averaging its word vectors.
    Words not in the vocabulary are silently skipped.
    """
    tokens = sentence.lower().split() # tokenise

    vectors = []

    for token in tokens:
        if token in model: # only use words we have vectors for
            vectors.append(model[token]) # model[token] returns a 100-D arr

    if not vectors: # if ALL words are unknown
        return np.zeros(dim) # return a zero vector as fallback

    return np.mean(vectors, axis=0) # element-wise average across all words

# Test it
s1 = 'man bite the dog at morning'
s2 = 'dog bite the man at afternoon'
s3 = 'A romantic comedy about falling in love in Paris'

v1 = sentence_vector(s1, glove)
v2 = sentence_vector(s2, glove)
v3 = sentence_vector(s3, glove)

# Compare similarity
from numpy.linalg import norm

cos = lambda a, b: np.dot(a, b) / (norm(a) * norm(b)) # cosine similarity

print(f's1 vs s2 (both action): {cos(v1, v2):.3f}') # high ~0.93
print(f's1 vs s3 (action vs romance): {cos(v1, v3):.3f}') # lower ~0.61

# The model correctly identifies that s1 and s2 are more similar!

```

```
s1 vs s2 (both action): 0.998
s1 vs s3 (action vs romance): 0.823
```

Using sentence vectors in a classifier

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import pandas as pd
# df has columns: 'description' (text) and 'genre' (label)

df = pd.DataFrame({
    "description": [
        "A brave warrior saves humanity from destruction",
        "A hero fights evil forces to protect the world",
        "Two strangers fall in love in Paris",
        "A romantic story about relationships and emotions",
        "A hilarious comedy full of funny situations",
        "Friends experience a series of humorous adventures",
        "A detective investigates a mysterious murder case",
        "A criminal conspiracy shocks the entire city"
    ],
    "genre": [
        "action",
        "action",
        "romance",
        "romance",
        "comedy",
        "comedy",
        "crime",
        "crime"
    ]
})
# Build one 100-D vector per movie description
# This replaces TF-IDF – instead of word counts, we use semantic meaning
X = np.array([sentence_vector(desc, glove) for desc in df['description']])
y = df['genre'] # e.g. 'action', 'romance', 'comedy'
# Standard train/test split
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.5, random_state=42, stratify=y
)
# Logistic Regression on semantic vectors
clf = LogisticRegression(max_iter=1000, C=1.0)
clf.fit(X_tr, y_tr)
print(classification_report(y_te, clf.predict(X_te)))
# Embedding-based model beats TF-IDF on unseen vocabulary!
```

	precision	recall	f1-score	support
action	1.00	1.00	1.00	1
comedy	1.00	1.00	1.00	1
crime	1.00	1.00	1.00	1
romance	1.00	1.00	1.00	1

accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

✓ Lab — Movie Description Similarity

Build a mini recommendation engine for Bollywood & Hollywood movies.

Part A — Create the movie dataset

```
import numpy as np
import pandas as pd
from gensim.models import KeyedVectors
# Load pre-trained GloVe (100 dimensions)
glove = KeyedVectors.load_word2vec_format('glove.6B.100d.w2v.txt',
binary=False)
# Our movie dataset — Bollywood + Hollywood
movies = [
# Bollywood
{'title':'Dilwale Dulhania Le Jayenge',
'desc':'A young man follows the woman he loves to India after they fall inlo
{'title':'Lagaan',
'desc':'Villagers challenge British colonisers to a cricket match to avoid pay
{'title':'Dangal',
'desc':'A father trains his daughters to become world class wrestlers against
{'title':'3 Idiots',
'desc':'Three college friends question the pressure of engineering
education in India'},
{'title':'Dil Chahta Hai',
'desc':'Three best friends navigate love relationships and life after
graduation'},
# Hollywood
{'title':'Titanic',
'desc':'A wealthy girl and a poor artist fall in love on a doomed ocean
liner'},
{'title':'Rocky',
'desc':'An underdog boxer trains relentlessly for a shot at the world
heavyweight champion'},
{'title':'Good Will Hunting',
'desc':'A genius janitor from Boston works through emotional trauma with a
therapist'},
{'title':'Chariots of Fire',
'desc':'Two British athletes train and compete for glory in the 1924
Olympic Games'},
{'title':'Before Sunrise',
'desc':'Two strangers meet on a train in Europe and spend one romantic
night in Vienna'},
]
```

```
df = pd.DataFrame(movies)
print(df[['title']].to_string())
```

Part B — Build vectors and find similar movies

```
from sklearn.metrics.pairwise import cosine_similarity
def sentence_vector(text, model, dim=100):
    tokens = text.lower().split()
    vecs = [model[t] for t in tokens if t in model]
    return np.mean(vecs, axis=0) if vecs else np.zeros(dim)
# Build vector for each movie description
df['vector'] = df['desc'].apply(lambda d: sentence_vector(d, glove))
# Stack all vectors into a matrix (10 x 100)
V = np.stack(df['vector'].values)
# Function: given a query description, find top-N most similar movies
def find_similar(query_desc, df, top_n=3):
    # Vectorise the query using the same sentence_vector function
    qvec = sentence_vector(query_desc, glove).reshape(1, -1)
    # Compute cosine similarity between query and every movie
    sims = cosine_similarity(qvec, V)[0] # returns array of 10 scores
    # Get indices sorted from highest to lowest similarity
    idx = np.argsort(sims)[::-1][:top_n]
    result = df.iloc[idx][['title']].copy()
    result['similarity'] = np.round(sims[idx], 3)
    return result
# --- Query 1: underdog sports story ---
q1 = 'An underdog athlete trains hard and wins against all expectations'
print('QUERY:', q1)
print(find_similar(q1, df).to_string(index=False))
# Expected: Dangan, Rocky, Chariots of Fire near the top
# --- Query 2: romance in Europe ---
q2 = 'Two people fall in love while travelling across Europe'
print('\nQUERY:', q2)
print(find_similar(q2, df).to_string(index=False))
# Expected: DDLJ, Before Sunrise, Titanic near the top
```

Part C — Visualise movie clusters

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
# Reduce 100-D vectors to 2D for visualisation
pca = PCA(n_components=2, random_state=42)
coords = pca.fit_transform(V) # shape: (10, 2)
# Colour Bollywood purple, Hollywood teal
colours = ['#7F77DD']*5 + ['#1D9E75']*5
fig, ax = plt.subplots(figsize=(11, 7))
ax.scatter(coords[:,0], coords[:,1], c=colours, s=120, zorder=3)
# Label each point with the movie title
for i, row in df.iterrows():
    ax.annotate(row['title'], coords[i], fontsize=8.5,
                xytext=(6, 4), textcoords='offset points')
# Add legend
```

```
import matplotlib.patches as mpatches
ax.legend(handles=[
    mpatches.Patch(color='#7F77DD', label='Bollywood'),
    mpatches.Patch(color='#1D9E75', label='Hollywood'),
], fontsize=10)
ax.set_title('Movie description clusters (GloVe + PCA)', fontsize=13)
plt.tight_layout()
plt.savefig('movie_clusters.png', dpi=150)
plt.show()
# Action movies (Lagaan, Rocky, Dangal, Chariots) should cluster together
# Romance movies (DDLJ, Titanic, Before Sunrise) should be in another
cluster
```

✓ Interview Questions

Q1. What is a word embedding and how is it different from one-hot encoding?

Δ• One-hot encoding assigns a unique binary vector per word (huge, sparse, no